

Função

unpack_telemetria_completa

```
def unpack_telemetria_completa(decrypted_data):
    try:
        # Lê a identificação
        net_id = (decrypted_data[0] >> 5) & 0x07
        vest_id = decrypted_data[0] & 0x1F
        flags = decrypted_data[1]

        # Lê 2 Inteiros (Lat/Lon), 1 Short (Alt) e 1 Byte
        # (BPM)
        lat_raw, long_raw, alt_raw, bpm = struct.unpack('>i
i h B', decrypted_data[2:13])

        # Isola os dois últimos bytes para processar o oxig
        # énio e temperatura
        b13 = decrypted_data[13]
        b14 = decrypted_data[14]

        spo2_raw = (b13 >> 3) & 0x1F
        spo2 = (spo2_raw + 69) if spo2_raw > 0 else 0

        temp_raw = ((b13 & 0x07) << 8) | b14
        temperature = 25.00 + (temp_raw / 100.0)

        return {
            "tipo": "COMPLETA",
            "vest_id": vest_id,
            "bpm": bpm,
            "spo2": spo2,
            "temperature": temperature,
            "mandown": bool((flags >> 5) & 0x01),
```

```

        "sos": bool((flags >> 6) & 0x01),
        "lat_raw": lat_raw,
        "lon_raw": long_raw
    }
except Exception as e:
    return None

```

```

        net_id = (decrypted_data[0] >> 5) & 0x07
        vest_id = decrypted_data[0] & 0x1F
        flags = decrypted_data[1]

```

- O **byte 0** tem a identificação, com os comandos `>> 5` e `& 0x07` puxamos os 3 bits mais à esquerda e lemos o `net_id`
- Com o comando `& 0x1F`, lemos apenas os 5 bits da direita para sabermos o `vest_id`
- O **byte 1** tem os alertas (Flags). Logo guardamos o byte inteiro na variável `flags`

```

lat_raw, long_raw, alt_raw, bpm = struct.unpack('>ii h B',
decrypted_data[2:13])

```

- O **struct.unpack()**, lê os 11 bytes de uma só vez (do byte 2 ao byte 12)
- **>ii h B**: A leitura é feita da esquerda para a direita (Big_Endian) e são lidos 2 inteiros de 4 bytes cada(ii), sendo 4 bytes para a latitude e 4 bytes para a longitude. São lidos também 1 short de 2 bytes(altitude) e 1 byte sem sinal (bpm)

```

spo2_raw = (b13 >> 3) & 0x1F
spo2 = (spo2_raw + 69) if spo2_raw > 0 else 0

```

- O intervalo de SpO2 definido foi entre os 70% e 100%. Por isso, não envia o número "98", envia apenas a diferença acima de 69.

- **b13 >> 3**: Corta os 5 bits mais à esquerda do byte 13.
- **+ 69**: Se o rádio enviou o número "30", o código soma 69 e descobre que o bombeiro tem 99% de oxigénio! (Isto permite guardar o SpO2 usando apenas 5 bits em vez de 8).

```
temp_raw = ((b13 & 0x07) << 8) | b14
temperature = 25.00 + (temp_raw / 100.0)
```

- Sobraram 3 bits no **b13** (acima só lemos 5), e temos os 8 bits completos do **b14** (11 bits no total).
- **((b13 & 0x07) << 8) | b14**: Pegamos nos 3 bits que sobraram no byte 13, empurramo-os para a esquerda, e juntamos ao byte 14. Com isto ficamos com um número inteiro de 11 bits.
- **25.00 + (temp_raw / 100.0)**: A temperatura corporal de um bombeiro vivo nunca é zero. A temperatura base é 25.00°C. Se o **temp_raw** for 1150, dividesse por 100 (dá 11.50) e somamos os 25. Ficamos ent com 36.50°C.

```
return {
    "tipo": "COMPLETA",
    "vest_id": vest_id,
    "bpm": bpm,
    "spo2": spo2,
    "temperature": temperature,
    "mandown": bool((flags >> 5) & 0x01),
    "sos": bool((flags >> 6) & 0x01),
    "lat_raw": lat_raw,
    "lon_raw": long_raw
}
```

- **"mandown"** e **"sos"**, vão buscar o **Byte 1** que tem as **flags** que guardámos antes, olham para o bit 5 e para o bit 6 (atraves dos comandos **>>** e a máscara **& 0x01**) e transformam-nos em verdadeiro ou falso (**bool**). Se o bit for 1, significa que o há alerta.

